

PYTHON AND DATA STRUCTURES FOUNDATIONS

1. INTRODUCTION TO PYTHON

Python is a high-level programming language known for its simplicity and readability. It is widely used in web development, machine learning, automation, and data science.

Python code is executed line by line by the Python interpreter. Variables in Python are dynamically typed, meaning you do not need to specify the data type explicitly.

Example:

```
x = 10
name = "Santhosh"
```

Python supports multiple programming paradigms including procedural programming and object-oriented programming.

2. FUNCTIONS IN PYTHON

Functions are reusable blocks of code designed to perform specific tasks. They help organize programs into smaller and manageable sections.

A function is defined using the `def` keyword.

Example:

```
def greet():
    print("Hello")
```

Functions can accept parameters and return values.

Example:

```
def add(a, b):
    return a + b
```

Functions reduce code duplication and improve readability.

3. RECURSION

Recursion is a programming technique where a function calls itself repeatedly until a base condition is met.

Every recursive function must contain:

1. Base case
2. Recursive case

Example:

```
def factorial(n):  
  
    if n == 1:  
        return 1  
  
    return n * factorial(n - 1)
```

Recursion is commonly used in:

- tree traversal
- divide and conquer algorithms
- backtracking problems

The function calls are stored in memory using the call stack.

4. STACK DATA STRUCTURE

A stack is a linear data structure that follows the Last In First Out (LIFO) principle.

Imagine stacking plates on top of each other. The last plate placed on top is the first plate removed.

Basic stack operations:

- push()
- pop()
- peek()

Stacks are used in:

- recursion
- expression evaluation
- undo operations
- browser history

Recursive function calls internally use the call stack.

5. QUEUE DATA STRUCTURE

A queue is a linear data structure that follows the First In First Out (FIFO) principle.

Imagine a queue at a ticket counter. The first person entering the line is served first.

Basic queue operations:

- enqueue()
- dequeue()

Queues are used in:

- scheduling
- task management
- breadth-first search
- buffering systems

6. LISTS IN PYTHON

Lists are ordered and mutable collections in Python.

Example:

```
numbers = [1, 2, 3, 4]
```

Lists can store multiple data types.

Common list operations:

- append()
- remove()
- insert()
- pop()

Lists are frequently used to implement stacks and queues.

7. DICTIONARIES IN PYTHON

Dictionaries store data using key-value pairs.

Example:

```
student = {  
    "name": "Santhosh",  
    "age": 20  
}
```

Dictionaries provide fast lookup operations because they use hashing internally.

They are useful in:

- caching
- indexing
- databases
- frequency counting

8. TIME COMPLEXITY

Time complexity measures how the runtime of an algorithm grows relative to input size.

Big O notation is commonly used.

Examples:

- $O(1)$ → constant time
- $O(n)$ → linear time
- $O(\log n)$ → logarithmic time
- $O(n^2)$ → quadratic time

Understanding time complexity helps developers write efficient algorithms.

9. SEARCHING ALGORITHMS

Linear Search:

Checks each element one by one.

Time Complexity:

$O(n)$

Binary Search:

Works only on sorted arrays.

Repeatedly divides the search space into half.

Time Complexity:

$O(\log n)$

Binary search is significantly faster for large sorted datasets.

10. OBJECT-ORIENTED PROGRAMMING

Object-oriented programming (OOP) organizes code using objects and classes.

A class is a blueprint for creating objects.

Example:

class Student:

```
def __init__(self, name):  
    self.name = name
```

Objects help model real-world entities and improve code modularity.

Core concepts of OOP:

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction